위상수학적 음악 분석 — Step 2

알고리즘 의사코드 및 복잡도 분석

본 장에서는 본 연구의 두 가지 음악 생성 알고리즘 — **Algorithm 1** (확률적 샘플링) 과 **Algorithm 2** (신경망 기반 시퀀스 모델) — 을 의사코드로 정형화하고, 각 알고리즘의 시간/공간 복잡도를 분석한다.

표기 정의

본 장에서 사용할 표기를 다음과 같이 통일한다.

| 기호 | 의미 | hibari 값 |

|------|------|----------|

| T | 시간축 길이 (8분음표 단위) | $1{,}088$ |

| N | 고유 note 수 ($(\text{pitch}, \text{duration})$ 쌍) | 23 |

| C | 발견된 cycle 수 | 최대 48 |

| K | 선택된 cycle subset 크기 ($K \leq C$) | $\{10, 17, 48\}$ |

| $O \in \{0,1\}^{T \times K}$ | 중첩행렬 (overlap matrix) | — |

| L_t | 시점 t에서 추출할 note 개수 | $\sum_t L_t \approx 165$ |

| $V(c)$ | cycle c의 vertex (note label) 집합 | $\lvert V(c) \rvert \in \{4, 5, 6\}$ |

| R | 재샘플링 최대 시도 횟수 | 50 |

| B | 학습 미니배치 크기 | 32 |

| E | 학습 epoch 수 | 200 |

| H | DL 모델의 hidden dimension | 128 |

2.10 Algorithm 1 — 확률적 샘플링 기반 음악 생성

알고리즘 개요

Algorithm 1은 **중첩행렬의 ON/OFF 패턴**을 직접 참조하여, 각 시점에서 활성화된 cycle들이 공통으로 포함하는 note pool로부터 확률적으로 음을 추출하는 결정론적 규칙 기반 알고리즘이다. 신경망 학습 없이 즉시 생성이 가능하며, 중첩행렬이 곧 "구조적 seed" 역할을 한다.

핵심 아이디어 (3가지 규칙)

1. **시점 t에서 활성 cycle이 있는 경우** ($\sum_c O[t,c] > 0$):

활성 cycle들의 vertex 집합의 **교집합** $\bigcap_{c:\, O[t,c]=1} V(c)$ 에서 note를 균등 추출한다. 교집합이 비어 있으면 전체 note pool에서 추출한다.

2. **시점 t에서 활성 cycle이 없는 경우** ($\sum_c O[t,c] = 0$):

인접 시점($t-1$, $t+1$)의 활성 cycle의 vertex **합집합**을 피해서 추출한다. 이는 "위상 구조 외 영역"에서 노이즈가 위상 구조를 침범하지 않도록 보호한다.

3. **중복 onset 방지:**

같은 시점 t에서 동일한 $(\text{pitch}, \text{duration})$이 두 번 추출되지 않도록 `onset_checker`로 검사하며, 충돌 시 최대 R 회까지 재샘플링한다.

의사코드

```
Algorithm 1: TopologySampleGen
```

```
Input:
   O   {0,1}^{T×K}      (overlap matrix)
   {V(c)}_{c=1..K}       (cycle vertex sets)
   L = (L_1, ..., L_T)   (target note count per timepoint)
   P                     (full note pool, |P| = N)
   R                     (max resample attempts)
Output:
   G = [(start, pitch, end), ...]   (generated note list)

 1: G ←
 2: onset_checker[t] ←    for t = 1..T
 3: for t = 1 to T do
 4:     n_to_sample ← L_t
 5:     for i = 1 to n_to_sample do
 6:         flag ← Σ_c O[t, c]
 7:         for attempt = 1 to R do
 8:             if flag > 0 then
 9:                 I ←  _{c : O[t,c]=1} V(c)
10:                 if I ≠   then
11:                     z ← UniformSample(I)
12:                 else
13:                     z ← UniformSample(P)
14:             else
15:                 U_prev ←  _{c : O[t-1,c]=1} V(c)    if t > 1
16:                 U_next ←  _{c : O[t+1,c]=1} V(c)    if t < T
17:                 avoid ← U_prev   U_next
18:                 z ← UniformSample(P \ avoid)
19:             (pitch, duration) ← LabelToInfo(z)
20:             end ← min(t + duration, T)
21:             if (pitch, end - t)   onset_checker[t] then
22:                 break
23:         if attempt = R then
24:             continue                                  failure, skip
25:         G.append((t, pitch, end))
26:         onset_checker[t].add((pitch, end - t))
27:         for s = t + 1 to min(end, T) do
28:             L_s ← max(0, L_s - 1)                    duration
29:             onset_checker[s].add((pitch, end - t))
30: return G
```

시간 복잡도 분석

각 시점 t에서의 작업을 단계별로 보면:

- **교집합 계산** (line 9): 활성 cycle 수를 $k_t = \sum_c O[t,c]$이라 하면, k_t개의 vertex 집합 (각각 평균 $|V| \approx 5$) 의 교집합은 $O(k_t \cdot |V|)$.
- **합집합 계산** (line 15-16): 마찬가지로 $O(k_t \cdot |V|)$.
- **샘플링과 검사** (line 11-22): 한 시도당 $O(|V|+1) = O(|V|)$, 최대 R회.
- **활성 구간 차감** (line 27-29): note duration의 평균을 $\bar{d}$라 하면 $O(\bar{d})$.

한 시점에서 L_t개를 추출하므로, 시점 t의 총 비용은

$$O(L_t \cdot (R \cdot |V| + k_t \cdot |V| + \bar{d})).$$

전체 합계는

$$T_{\text{Algo1}} = O(\sum_{t=1}^{T} L_t \cdot (R|V| + k_t|V| + \bar{d})) = O(\bar{L} \cdot T \cdot (R + \bar{k}) \cdot |V|).$$

여기서 $\bar{L}=\frac{1}{T}\sum_t L_t$, $\bar{k}=\frac{1}{T}\sum_t k_t$이다. 대표적인 hibari 파라미터 ($T=1088$, $\bar{L}\approx 0.15$, $R=50$, $\bar{k}\le K\le 48$, $|V|\approx 5$) 를 대입하면 약 4×10^4 회 연산으로 추정되며, **실측 약 50ms** 내에 끝난다.

핵심은 T_{Algo1}이 **선형 복잡도** $O(T)$로 스케일링되며, K (cycle 수) 의 영향은 상수배 $|V|$ 에 흡수되므로 K가 커져도 안정적으로 작동한다는 점이다.

공간 복잡도 분석

- 중첩행렬 O: $O(TK)$ bit
- `onset_checker`: 최악의 경우 $O(TN)$ entry
- cycle vertex 집합 $\{V(c)\}$: $O(K\cdot|V|)$
- 출력 note list G: $O(\sum_t L_t)=O(T)$

총 공간 복잡도는

$$S_{\text{Algo1}}=O(TK+TN+K|V|+T)=O(T(K+N)).$$

hibari의 경우 $T=1088$, $K\le 48$, $N=23$이므로 약 $80\,\text{KB}$ 수준이며, 메모리 병목이 아니다.

2.11 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

알고리즘 개요

Algorithm 2는 중첩행렬을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑 $f_\theta:\{0,1\}^C\to\mathbb{R}^N$ (또는 시퀀스 모델의 경우 $\{0,1\}^{T\times C}\to\mathbb{R}^{T\times N}$) 을 학습한다. 학습된 모델은 다른 cycle subset이나 노이즈가 섞인 중첩행렬에서도 위상 구조를 보존하는 음악을 복원할 수 있다.

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	파라미터 수 (대략)
FC	(B,C)	$\approx 4\times 10^4$
LSTM (2-layer)	(B,T,C)	$\approx 2\times 10^5$
Transformer (2-layer, 4-head)	(B,T,C)	$\approx 4\times 10^5$

학습 데이터 구성과 증강

$X\in\{0,1\}^{T\times C}$, $y\in\{0,1\}^{T\times N}$이 원본 학습 쌍이다. 본 연구는 세 가지 증강 전략을 적용하여 학습 데이터를 약 $7\sim 10$배 늘린다.

1. **Subset Augmentation:** $K\in\{10,15,20,30\}$의 cycle subset에 대한 중첩행렬을 생성, 동일한 y에 매핑.
2. **Circular Shift:** X,y를 시간축으로 동일하게 회전 (위치 편향 방지).
3. **Noise Injection:** X에 확률 $p=0.03$로 bit flip (overfitting 방지).

의사코드 (학습 단계)

```
Algorithm 2-A: TopologyToMusic-Train
──────────────────────────────────────────────────────────
Input:
   O   {0,1}^{T×C}                  (overlap matrix, full)
   M = (M_1, M_2)                    (instrument 1, 2 note lists)
   notes_label : (pitch, dur) → int  (1-indexed labels)
   E, B, η                           (epochs, batch size, learning rate)
   Model   {FC, LSTM, Transformer}
Output:
   trained model parameters θ*
```

```
y ← BuildOnehotMatrix(M, notes_label, T, N)          (T, N)
X ← O.astype(float32)                                (T, C)
(X_aug, y_aug) ← AugmentData(X, y, O, ...)            (T_aug, ·)
f_θ ← Model(num_cycles=C, num_notes=N, ...)
optimizer ← Adam(f_θ.parameters(), lr=η)
pos_weight ← (#negatives / #positives)                class imbalance
for epoch = 1 to E do
    shuffle (X_aug, y_aug)
    for each minibatch (X_b, y_b) of size B do
        ŷ_b ← f_θ(X_b)                                raw logits
        L ← BCEWithLogitsLoss(ŷ_b, y_b, pos_weight)
        L.backward()
        optimizer.step()
        optimizer.zero_grad()
return θ*
```

여기서 `BCEWithLogitsLoss`는 2.8절에서 정의한 multi-label binary cross-entropy 손실에 sigmoid가 통합된 수치 안정 버전이다.

의사코드 (생성/추론 단계)

```
Algorithm 2-B: TopologyToMusic-Generate
──────────────────────────────────────────────────────────
Input:
   O_test   {0,1}^{T×C}              (overlap matrix to generate from)
   trained model f_θ
   notes_label
   ρ = 0.15                          (target ON ratio)
   gap_min                           (minimum onset gap, 8th-note units)
Output:
   G = [(start, pitch, end), ...]

f_θ.eval()
with no_grad:
    ẑ ← f_θ(O_test)                                   (T, N) logits
    P ← σ(ẑ)                                          sigmoid
θ_thr ← Quantile(P.flatten(), 1 - ρ)                 adaptive threshold
G ←
last_onset ← -gap_min
for t = 1 to T do
    if (t - last_onset) < gap_min then continue
    onset_made ← false
    for n = 1 to N do
        if P[t, n] ≥ θ_thr then
            (pitch, dur) ← LabelToInfo(n)
            G.append((t, pitch, t + dur))
            onset_made ← true
    if onset_made then last_onset ← t
return G
```

시간 복잡도 분석

(1) 학습 단계 (Algorithm 2-A). 한 forward+backward 패스의 비용은 모델 구조에 따라 다르다.

- **FC 모델 (시점별 독립 예측):** 한 시점당 $O(C \cdot H + H^2 + H \cdot N) = O(H^2)$ (지배항). 한 epoch는 T_{aug}개의 샘플을 처리하므로 $O(T_{\text{aug}} \cdot H^2 / B \cdot B) = O(T_{\text{aug}} \cdot H^2)$.
- **LSTM 모델 (2-layer, hidden H):** 한 시퀀스(T 시점) 처리에 $O(T \cdot H^2)$. T_{aug} 시점을 T 단위로 묶으므로 한 epoch는 $O(T_{\text{aug}} \cdot H^2)$.
- **Transformer (2-layer, T 시점, $d_{\text{model}} = H$):** Self-attention은 $O(T^2 \cdot H)$, feedforward는 $O(T \cdot H^2)$. 한 시퀀스 비용은 $O(T^2 H + T H^2)$, 한 epoch는 $O((T_{\text{aug}}/T) \cdot (T^2 H + T H^2)) = O(T_{\text{aug}} \cdot (T H + H^2))$.

따라서 전체 학습 시간은:

$$T_{\text{train}}^{\text{FC}} = O(E \cdot T_{\text{aug}} \cdot H^2)$$

$$T_{\text{train}}^{\text{LSTM}} = O(E \cdot T_{\text{aug}} \cdot H^2)$$

$$T_{\text{train}}^{\text{Trans}} = O(E \cdot T_{\text{aug}} \cdot (TH + H^2))$$

hibari의 경우 ($E = 200$, $T_{\text{aug}} \approx 8000$, $H = 128$, $T = 1088$) 실측 학습 시간은 FC가 약 30s, LSTM은 약 90s, Transformer는 약 180s (CPU 기준).

(2) 추론 단계 (Algorithm 2-B). 한 번의 forward 패스 비용에 quantile 계산 $O(TN\log(TN))$, 임계값 비교 $O(TN)$이 추가된다.

$$T_{\text{infer}} = O(\text{modelforward}) + O(TN\log(TN)) = O(TN\log(TN) + TH^2)$$

대표값 $T = 1088, N = 23, H = 128$에서 약 10^6 연산으로, 모든 모델에서 추론은 100ms 미만이다.

공간 복잡도 분석

모델 파라미터:

- FC: $O(CH + H^2 + HN) = O(H^2)$
- LSTM (2-layer): $O(CH + H^2 \cdot \text{layers}) = O(H^2)$
- Transformer (2-layer): $O(CH + H^2 \cdot \text{layers} + T \cdot H) = O(H^2 + TH)$

활성값 (학습 시 한 미니배치):

- FC: $O(B \cdot H)$
- LSTM: $O(B \cdot T \cdot H)$
- Transformer: $O(B \cdot T \cdot H + B \cdot T^2)$ (attention map 포함)

학습 데이터 (증강 후):

$$S_{\text{data}} = O(T_{\text{aug}} \cdot (C + N))$$

hibari의 경우 $T_{\text{aug}} \approx 8000$, $C \leq 48$, $N = 23$이므로 약 $2.3\,\text{MB}$. Transformer의 attention map은 한 시퀀스당 $T^2 \approx 1.18 \times 10^6$ float이므로 한 시퀀스에 약 $5\,\text{MB}$이며, 작은 배치 크기 $B = 32$로도 GPU/CPU 메모리 안에서 충분히 학습 가능하다.

2.12 두 알고리즘의 비교 요약

| 항목 | Algorithm 1 (Sampling) | Algorithm 2 (DL) |

|------|---------------------|---------------------|

| 학습 필요 여부 | (즉시 생성) | (E epoch) |

| 시간 복잡도 (생성) | $O(\bar{L} \cdot T \cdot R \cdot \|V\|)$ | $O(TH^2)$ |

| 공간 복잡도 | $O(T(K + N))$ | $O(H^2 + TH)$ |

| 결정성 | 확률적 (난수) | 학습 후 결정적 |

| 일반화 | 같은 곡 내에서만 | 다른 cycle subset에도 generalize |

| 위상 보존 | 직접 강제 (교집합) | 손실함수를 통해 간접 |

| hibari 실측 (생성 1회) | $\sim 50\text{ms}$ | $\sim 100\text{ms}$ (학습 후) |

| hibari 실측 (학습 포함) | — | $30\text{s} \sim 3\text{min}$ |

해석. Algorithm 1은 위상 정보를 직접 규칙으로 강제하므로 cycle 보존도 측면에서 가장 신뢰할 수 있는 baseline이다. 반면 Algorithm 2는 학습된 잠재 표현을 통해 부드러운 생성이 가능하며, 학습 데이터에 없는 cycle subset에 대해서도 합리적인 음악을 만들어낸다. 본 연구의 실험에서는 두 알고리즘이 상호 보완적임을 보였다 — Algorithm 1은 위상 보존도, Algorithm 2는 음악적 자연스러움에서 각각 우위를 보인다 (Step 4 실험 결과 참조).

참고문헌 (Step 2 추가분)

- Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735-1780.
- Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.
- Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. — Loop invariant 및 점근 복잡도 분석 표기법.